a. ChatGPT and Gemini
b. ChatGPT from windows application, Gemini from its website (https://gemini.google.com/)
c. Gemini
d.

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

using namespace std;

// Class representing the Sudoku grid and its rules
class SudokuBoard {
private:
    int grid[9][9];

public:
    SudokuBoard() {
        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                grid[r][c] = 0;
            }
        }
    }

    // Loads the puzzle from a file.
    // Returns true if successful, false if the file cannot be opened.
    bool loadFromFile(const string& filename) {
        ifstream file(filename);
        if (!file.is_open()) {
            return false;
        }

        char cell;
        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                if (file >> cell) {
                    if (cell == '_') {
                        grid[r][c] = 0;
                    } else if (cell >= '1' && cell <= '9') {
                        grid[r][c] = cell - '0';
                    }
                }
```

```
            }
        }
        file.close();
        return true;
    }

    // Prints the 9x9 grid to standard output
    void print() const {
        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                if (grid[r][c] == 0) {
                    cout << "_ ";
                } else {
                    cout << grid[r][c] << " ";
                }
            }
            cout << endl;
        }
        cout << endl;
    }

    // Checks if it's valid to place 'num' at grid[row][col]
    bool isSafe(int row, int col, int num) const {
        // Check row and column
        for (int x = 0; x < 9; x++) {
            if (grid[row][x] == num || grid[x][col] == num) {
                return false;
            }
        }

        // Check 3x3 sub-grid
        int startRow = row - row % 3;
        int startCol = col - col % 3;
        for (int r = 0; r < 3; r++) {
            for (int c = 0; c < 3; c++) {
                if (grid[r + startRow][c + startCol] == num) {
                    return false;
                }
            }
        }

        return true;
    }
```

```
    int getCell(int row, int col) const {
        return grid[row][col];
    }

    void setCell(int row, int col, int num) {
        grid[row][col] = num;
    }
};

// Class responsible for solving a given Sudoku board
class SudokuSolver {
private:
    SudokuBoard board;
    int solutionCount;

    // Recursive depth-first search with backtracking
    void solveRecursive(int row, int col) {
        // If we have reached the 9th row, the board is fully filled
        if (row == 9) {
            solutionCount++;
            cout << "Solution " << solutionCount << ":" << endl;
            board.print();
            return;
        }

        // If we reach the end of a column, move to the next row
        if (col == 9) {
            solveRecursive(row + 1, 0);
            return;
        }

        // If the current cell is already filled, move to the next column
        if (board.getCell(row, col) != 0) {
            solveRecursive(row, col + 1);
            return;
        }

        // Try placing digits 1-9 in the empty cell
        for (int num = 1; num <= 9; num++) {
            if (board.isSafe(row, col, num)) {
                board.setCell(row, col, num);
```

```
                // Recursively attempt to fill the rest of the board
                solveRecursive(row, col + 1);

                // Backtrack
                board.setCell(row, col, 0);
            }
        }
    }

public:
    SudokuSolver(const SudokuBoard& initialBoard) {
        board = initialBoard;
        solutionCount = 0;
    }

    // Initiates the solving process and prints the outcome
    void solve() {
        solutionCount = 0;
        solveRecursive(0, 0);

        if (solutionCount == 0) {
            cout << "No solution found" << endl << endl;
        }
    }
};

int main() {
    // List of puzzle files to process
    vector<string> puzzleFiles = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    for (const string& filename : puzzleFiles) {
        cout << "====================================" << endl;
        cout << "File: " << filename << endl;

        SudokuBoard board;
        if (!board.loadFromFile(filename)) {
            cout << "Error: Could not open " << filename << endl;
```

```
            continue;
        }

        cout << "Original Puzzle:" << endl;
        board.print();

        SudokuSolver solver(board);
        solver.solve();
    }

    return 0;
}
```

ChatGPT

```
// Assignment4.cpp
// Sudoku Solver using Object-Oriented Design and Recursive Backtracking

#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <cctype>

using namespace std;

// Represents a 9x9 Sudoku board.
class SudokuBoard {
private:
    static const int SIZE = 9;
    char grid[SIZE][SIZE];

public:
    // Initialize the board with blank cells.
    SudokuBoard() {
        for (int row = 0; row < SIZE; row++) {
            for (int col = 0; col < SIZE; col++) {
                grid[row][col] = '_';
            }
        }
    }
```

```
    // Read a Sudoku puzzle from a file.
    bool loadFromFile(const string& filename) {
        ifstream inputFile(filename);

        if (!inputFile.is_open()) {
            return false;
        }

        int row = 0;
        int col = 0;
        char value;

        while (inputFile >> value) {
            if (value == '\n' || value == '\r') {
                continue;
            }

            if (value == '_' || (value >= '1' && value <= '9')) {
                if (row >= SIZE) {
                    return false;
                }

                grid[row][col] = value;
                col++;

                if (col == SIZE) {
                    col = 0;
                    row++;
                }
            }
        }

        inputFile.close();

        // A valid puzzle file must contain exactly 81 cells.
        return row == SIZE && col == 0;
    }

    // Display the Sudoku board.
    void printBoard() const {
        for (int row = 0; row < SIZE; row++) {
            if (row % 3 == 0) {
                cout << "+-------+-------+-------+" << endl;
```

```
            }

            for (int col = 0; col < SIZE; col++) {
                if (col % 3 == 0) {
                    cout << "| ";
                }

                cout << grid[row][col] << ' ';
            }

            cout << "|" << endl;
        }

        cout << "+-------+-------+-------+" << endl;
    }

    // Find the next empty cell.
    bool findEmptyCell(int& emptyRow, int& emptyCol) const {
        for (int row = 0; row < SIZE; row++) {
            for (int col = 0; col < SIZE; col++) {
                if (grid[row][col] == '_') {
                    emptyRow = row;
                    emptyCol = col;
                    return true;
                }
            }
        }

        // No empty cells remain.
        return false;
    }

    // Check whether a digit can be placed in a particular cell.
    bool isValidPlacement(int row, int col, char digit) const {
        // Check the row.
        for (int c = 0; c < SIZE; c++) {
            if (grid[row][c] == digit) {
                return false;
            }
        }

        // Check the column.
        for (int r = 0; r < SIZE; r++) {
```

```
            if (grid[r][col] == digit) {
                return false;
            }
        }

        // Find the top-left corner of the 3x3 block.
        int blockRow = (row / 3) * 3;
        int blockCol = (col / 3) * 3;

        // Check the 3x3 block.
        for (int r = blockRow; r < blockRow + 3; r++) {
            for (int c = blockCol; c < blockCol + 3; c++) {
                if (grid[r][c] == digit) {
                    return false;
                }
            }
        }

        return true;
    }

    // Get the value stored at a particular cell.
    char getCell(int row, int col) const {
        return grid[row][col];
    }

    // Set the value stored at a particular cell.
    void setCell(int row, int col, char value) {
        grid[row][col] = value;
    }
};

// Solves Sudoku puzzles using recursive depth-first search.
class SudokuSolver {
private:
    int solutionCount;

    // Recursive backtracking algorithm.
    void solveRecursive(SudokuBoard& board) {
        int row;
        int col;

        // Find the next empty cell.
```

```
        if (!board.findEmptyCell(row, col)) {
            // No empty cells remain, so a solution has been found.
            solutionCount++;

            cout << "\nSolution " << solutionCount << ":" << endl;
            board.printBoard();

            return;
        }

        // Try every possible digit from 1 through 9.
        for (char digit = '1'; digit <= '9'; digit++) {
            // Only try digits that satisfy the Sudoku rules.
            if (board.isValidPlacement(row, col, digit)) {
                // Place the candidate digit.
                board.setCell(row, col, digit);

                // Recursively solve the remaining cells.
                solveRecursive(board);

                // Backtrack: undo the placement before trying
                // the next candidate digit.
                board.setCell(row, col, '_');
            }
        }
    }

public:
    // Constructor initializes the solution counter.
    SudokuSolver() {
        solutionCount = 0;
    }

    // Solve the puzzle and print every solution.
    void solve(SudokuBoard& board) {
        solutionCount = 0;

        solveRecursive(board);

        if (solutionCount == 0) {
            cout << "\nNo solution found." << endl;
        } else {
            cout << "\nTotal solutions found: "
```

```
                << solutionCount << endl;
        }
    }
};

// Main program processes all five Sudoku puzzle files.
int main() {
    const int NUMBER_OF_PUZZLES = 5;

    // Create the five required puzzle filenames.
    string filenames[NUMBER_OF_PUZZLES] = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    // Process each puzzle file.
    for (int i = 0; i < NUMBER_OF_PUZZLES; i++) {
        cout << "\n========================================" << endl;
        cout << "Puzzle File: " << filenames[i] << endl;
        cout << "========================================" << endl;

        // Create a board for the current puzzle.
        SudokuBoard board;

        // Load the puzzle from its file.
        if (!board.loadFromFile(filenames[i])) {
            cout << "Error: Could not read a valid 9x9 puzzle "
                 << "from " << filenames[i] << "." << endl;
            continue;
        }

        // Display the original puzzle.
        cout << "\nOriginal Puzzle:" << endl;
        board.printBoard();

        // Create a solver and find all solutions.
        SudokuSolver solver;

        cout << "\nSolving " << filenames[i] << "..." << endl;
        solver.solve(board);
```

```
    }

    return 0;
}
```

e.

Correctness - Both programs use the right method, recursive backtracking, and both check the row, column, and 3x3 block before placing a digit. Both print every solution if there is more than one, and both print a message when there is none. The main difference is how they handle a bad input file. Gemini fills any missing cells with blanks and says nothing, so a short or broken file still runs and may give a wrong answer. ChatGPT checks that the file has exactly 81 cells and shows an error if it does not, which is better at catching problems. Neither one checks if the starting puzzle already breaks the rules, like two 5s in one row.

Execution time - Both have the same worst case, O(9^k), where k is the number of empty cells. Each empty cell tries up to 9 digits, and each try goes one level deeper. Checking if a digit is safe takes O(1) in both, because it always looks at about 27 cells. The small difference is how they find the next empty cell. Gemini just moves to the next column, which is O(1). ChatGPT scans the board from the top on every call, which is up to 81 checks. That is still O(1) because the board size never changes, but ChatGPT does more work each call. Both use endl, which flushes the output each time and can slow down printing when there are many solutions.

Space complexity - Both store the grid in O(1) space, because it is always 81 cells. The recursion adds O(k) space, with at most about 81 levels. ChatGPT stores the grid as char (81 bytes) and passes the board by reference, so there is only one grid. Gemini stores int (324 bytes) and keeps its own copy of the board inside the solver, so it has two grids. Gemini also makes a recursive call for filled cells and row ends, so its recursion goes a little deeper than ChatGPT's. The difference is small, but ChatGPT uses less space.

Maintainability - Both programs use two clear classes, good names, const methods, and no global variables. ChatGPT is easier to hand off. It has a SIZE constant, a separate findEmptyCell method, a bordered grid print, an error message for bad files, and more comments. Gemini is shorter and simple to read, but it uses the number 9 in many places and has fewer comments. Both mix printing with the solving logic, and both hardcode the file names in main. Both could use more comments and need proper prologue comments.

f.

ChatGPT version

i. It checks the puzzle file before solving. It makes sure the file has exactly 81 cells and stops with a clear error message if not. Gemini fills missing cells with blanks and says nothing, so a broken file can give a wrong answer without any warning.

ii. It uses less memory. It stores the grid as char (81 bytes) instead of int (324 bytes). It also passes the board by reference, so there is only one grid and no extra copy inside the solver.

iii. It only makes a recursive call when it finds an empty cell. Gemini also makes calls for filled cells and row ends, so its recursion has extra levels that ChatGPT does not need.

iv. The code is easier to read, debug, and extend later. It uses a SIZE constant instead of the number 9 everywhere. It has a separate findEmptyCell method, a bordered grid print that is easier to check, and more comments.

g.

Correctness - The program used to accept any starting puzzle, even one that already breaks the rules, and then search for a solution that cannot exist. I added a check before solving. If a digit is repeated in the same row, column, or block, the program now shows a clear error message and skips that puzzle.

Execution Time - The program used to scan the board from the top every time it looked for the next empty cell. Now findEmptyCell starts from the position of the last cell that was filled, so it does not repeat the same checks. I also changed endl to “\n”, so the output is not flushed after every line. So now output is sent in bigger groups instead of flushing and output still looks same. The method is still the same brute force search, so the worst case is still O(9^k).

Space Complexity - Nothing needed here. The grid is always 81 cells and is already stored as small chars. The board is passed by reference, so there is only one copy, and the recursion is at most about 81 levels deep.

Maintainability - Already solid, thanks to the file check, the SIZE constant, and the separate findEmptyCell method. The only small change is moving the file names out of main into one constant list, so they are easy to find and change later.